



<http://www.pieriandata.com>

Distribution Plots

Let's discuss some plots that allow us to visualize the distribution of a data set. These plots are:

- distplot
- jointplot
- pairplot
- rugplot
- kdeplot

Imports

```
In [2]: import seaborn as sns  
%matplotlib inline
```

Data

Seaborn comes with built-in data sets!

```
In [3]: tips = sns.load_dataset('tips')
```

```
In [4]: tips.head()
```

```
Out[4]:
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

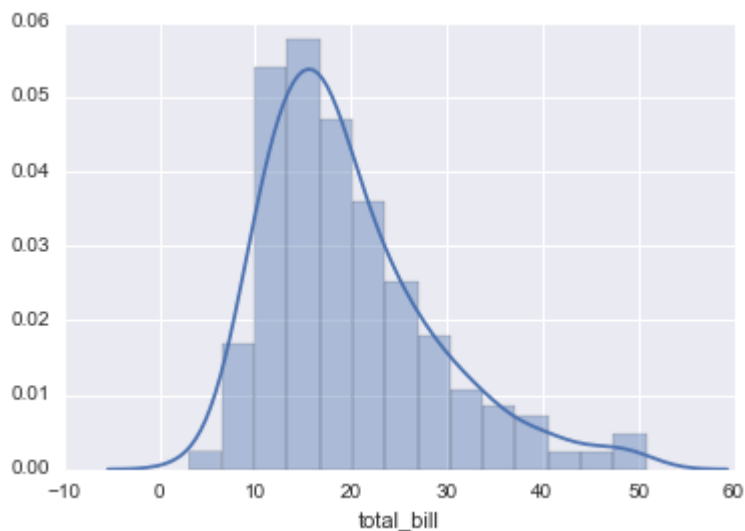
distplot

The distplot shows the distribution of a univariate set of observations.

```
In [16]: sns.distplot(tips['total_bill'])
# Safe to ignore warnings
```

```
/Users/marci/anaconda/lib/python3.5/site-packages/statsmodels/nonparametric/kde
tools.py:20: VisibleDeprecationWarning: using a non-integer number instead of a
n integer will result in an error in the future
y = X[:m/2+1] + np.r_[0,X[m/2+1:],0]*1j
```

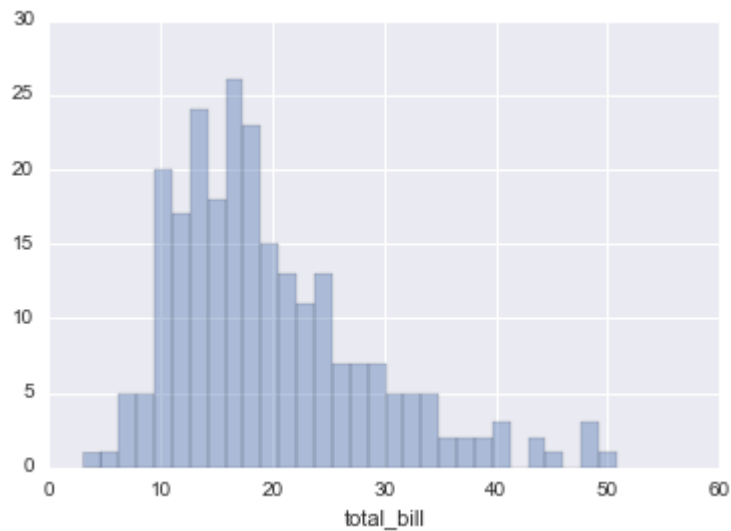
```
Out[16]: <matplotlib.axes._subplots.AxesSubplot at 0x11dd8e5f8>
```



To remove the kde layer and just have the histogram use:

```
In [9]: sns.distplot(tips['total_bill'],kde=False,bins=30)
```

```
Out[9]: <matplotlib.axes._subplots.AxesSubplot at 0x11c7b8668>
```



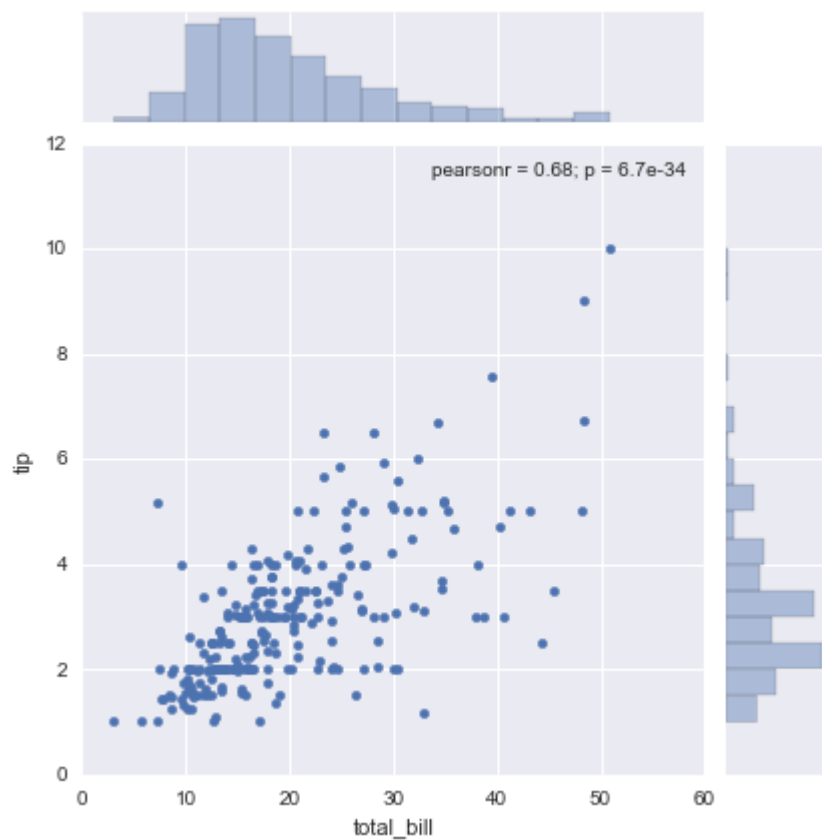
jointplot

jointplot() allows you to basically match up two distplots for bivariate data. With your choice of what **kind** parameter to compare with:

- “scatter”
- “reg”
- “resid”
- “kde”
- “hex”

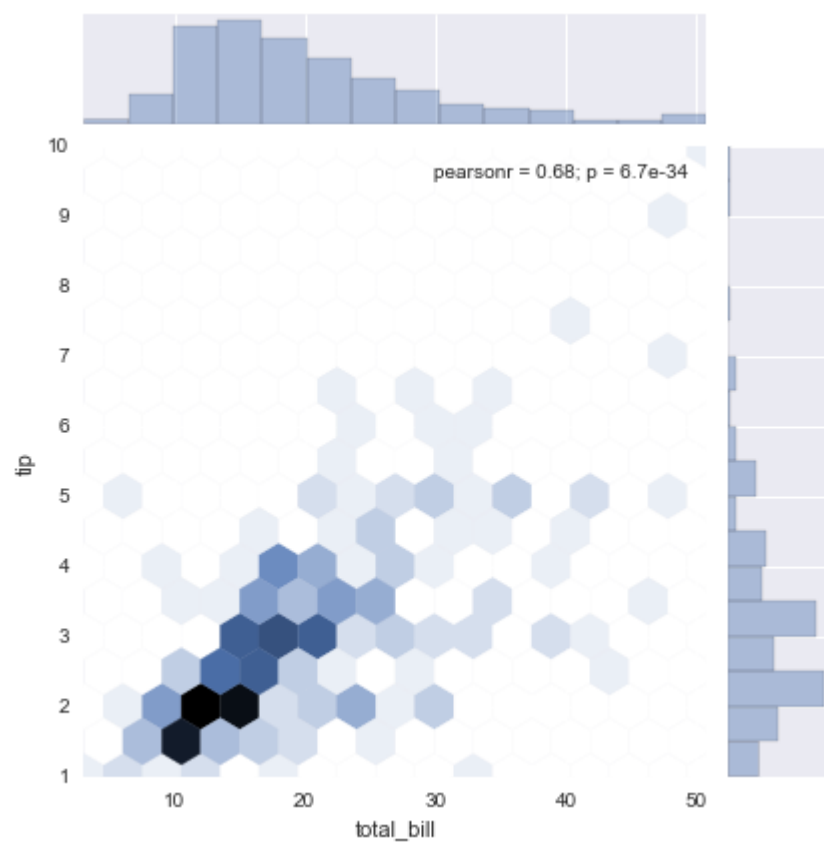
```
In [12]: sns.jointplot(x='total_bill',y='tip',data=tips,kind='scatter')
```

```
Out[12]: <seaborn.axisgrid.JointGrid at 0x11cfb28d0>
```



```
In [15]: sns.jointplot(x='total_bill',y='tip',data=tips,kind='hex')
```

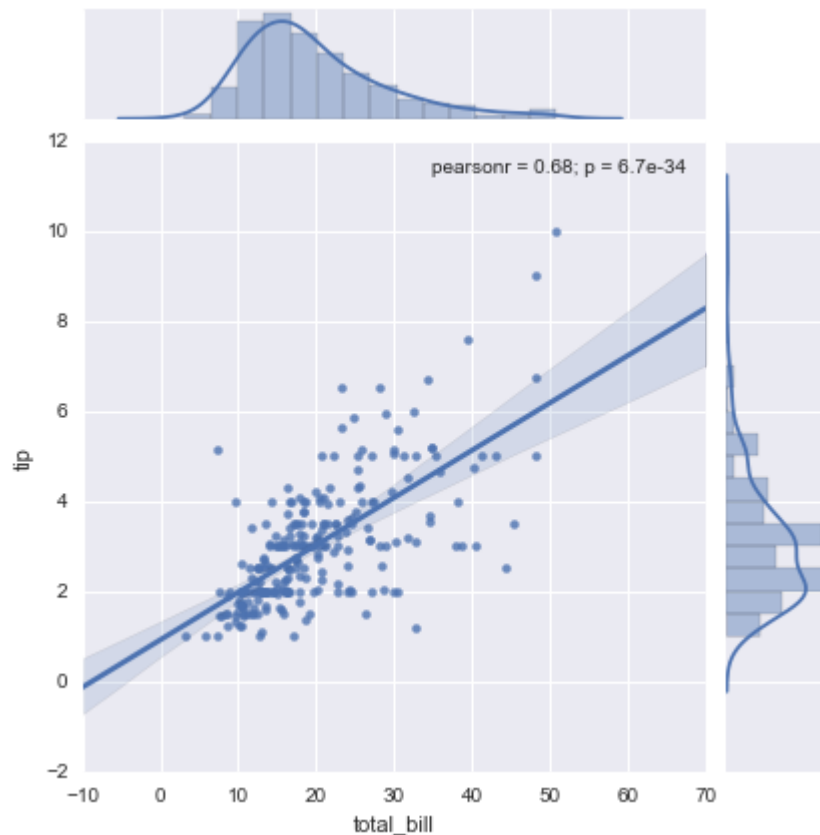
```
Out[15]: <seaborn.axisgrid.JointGrid at 0x11d96f160>
```



```
In [17]: sns.jointplot(x='total_bill',y='tip',data=tips,kind='reg')
```

```
/Users/marci/anaconda/lib/python3.5/site-packages/statsmodels/nonparametric/kde  
tools.py:20: VisibleDeprecationWarning: using a non-integer number instead of a  
n integer will result in an error in the future  
y = X[:m/2+1] + np.r_[0,X[m/2+1:],0]*1j
```

```
Out[17]: <seaborn.axisgrid.JointGrid at 0x11e0cfba8>
```

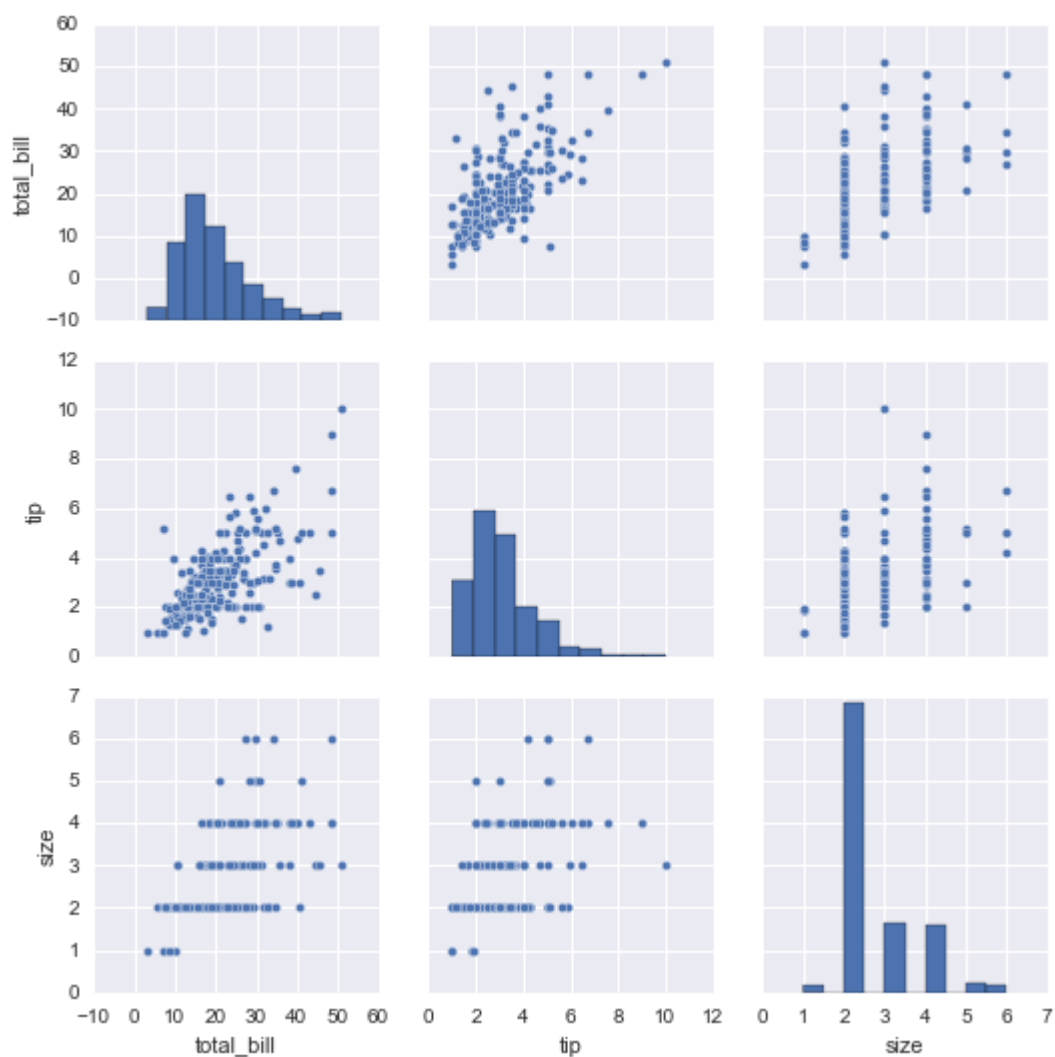


pairplot

`pairplot` will plot pairwise relationships across an entire dataframe (for the numerical columns) and supports a color hue argument (for categorical columns).

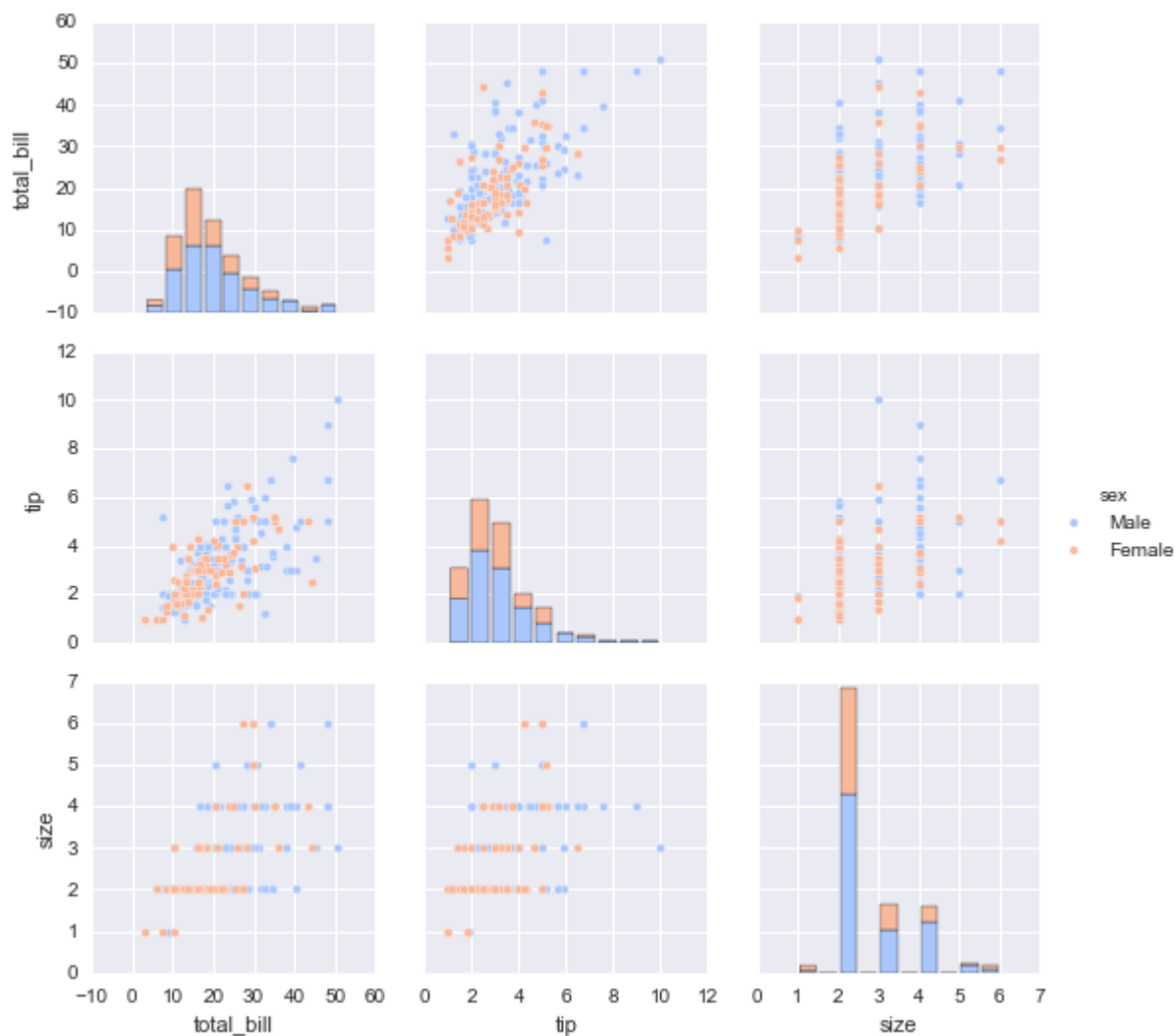
```
In [18]: sns.pairplot(tips)
```

```
Out[18]: <seaborn.axisgrid.PairGrid at 0x11e844208>
```



```
In [21]: sns.pairplot(tips,hue='sex',palette='coolwarm')
```

```
Out[21]: <seaborn.axisgrid.PairGrid at 0x11ff7a828>
```

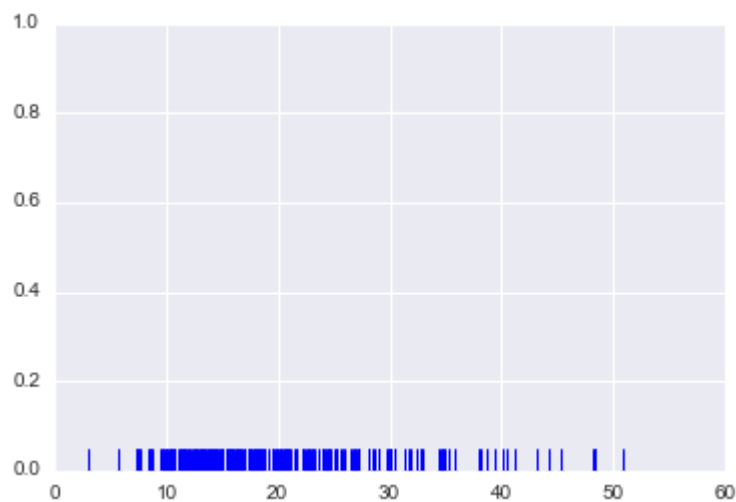


rugplot

rugplots are actually a very simple concept, they just draw a dash mark for every point on a univariate distribution. They are the building block of a KDE plot:


```
In [22]: sns.rugplot(tips['total_bill'])
```

```
Out[22]: <matplotlib.axes._subplots.AxesSubplot at 0x1207c8b70>
```



kdeplot

kdeplots are [Kernel Density Estimation plots](http://en.wikipedia.org/wiki/Kernel_density_estimation#Practical_estimation_of_the_bandwidth)

(http://en.wikipedia.org/wiki/Kernel_density_estimation#Practical_estimation_of_the_bandwidth).

These KDE plots replace every single observation with a Gaussian (Normal) distribution centered around that value. For example:

```
In [35]: # Don't worry about understanding this code!
# It's just for the diagram below
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

#Create dataset
dataset = np.random.randn(25)

# Create another rugplot
sns.rugplot(dataset);

# Set up the x-axis for the plot
x_min = dataset.min() - 2
x_max = dataset.max() + 2

# 100 equally spaced points from x_min to x_max
x_axis = np.linspace(x_min,x_max,100)

# Set up the bandwidth, for info on this:
url = 'http://en.wikipedia.org/wiki/Kernel\_density\_estimation#Practical\_estimation'

bandwidth = ((4*dataset.std()**5)/(3*len(dataset)))**.2

# Create an empty kernel list
kernel_list = []

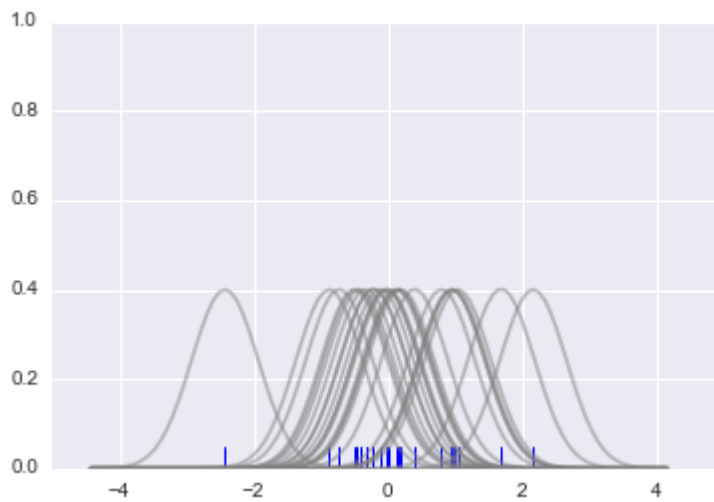
# Plot each basis function
for data_point in dataset:

    # Create a kernel for each point and append to list
    kernel = stats.norm(data_point,bandwidth).pdf(x_axis)
    kernel_list.append(kernel)

    #Scale for plotting
    kernel = kernel / kernel.max()
    kernel = kernel * .4
    plt.plot(x_axis,kernel,color = 'grey',alpha=0.5)

plt.ylim(0,1)
```

Out[35]: (0, 1)



In [37]: *# To get the kde plot we can sum these basis functions.*

```
# Plot the sum of the basis function
sum_of_kde = np.sum(kernel_list,axis=0)

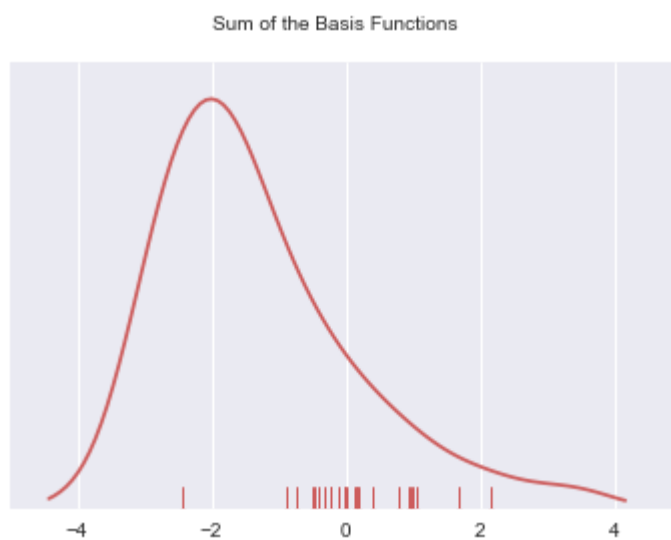
# Plot figure
fig = plt.plot(x_axis,sum_of_kde,color='indianred')

# Add the initial rugplot
sns.rugplot(dataset,c = 'indianred')

# Get rid of y-tick marks
plt.yticks([])

# Set title
plt.suptitle("Sum of the Basis Functions")
```

Out[37]: <matplotlib.text.Text at 0x121c41da0>

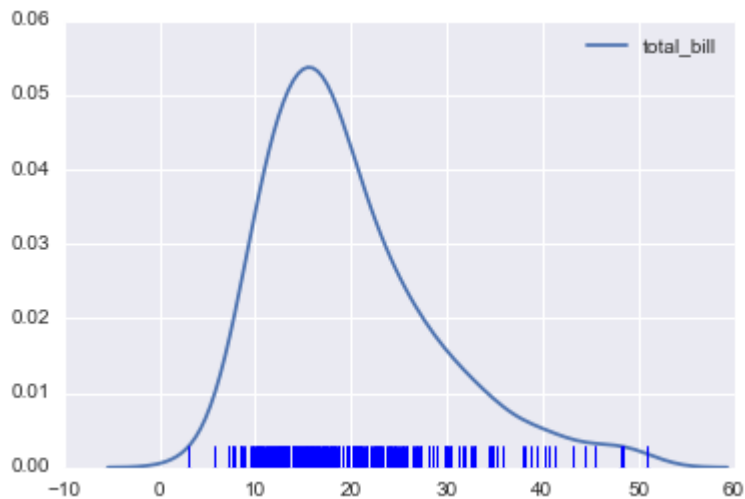


So with our tips dataset:

```
In [41]: sns.kdeplot(tips['total_bill'])  
sns.rugplot(tips['total_bill'])
```

```
/Users/marci/anaconda/lib/python3.5/site-packages/statsmodels/nonparametric/kde  
tools.py:20: VisibleDeprecationWarning: using a non-integer number instead of a  
n integer will result in an error in the future  
y = X[:m/2+1] + np.r_[0,X[m/2+1:],0]*1j
```

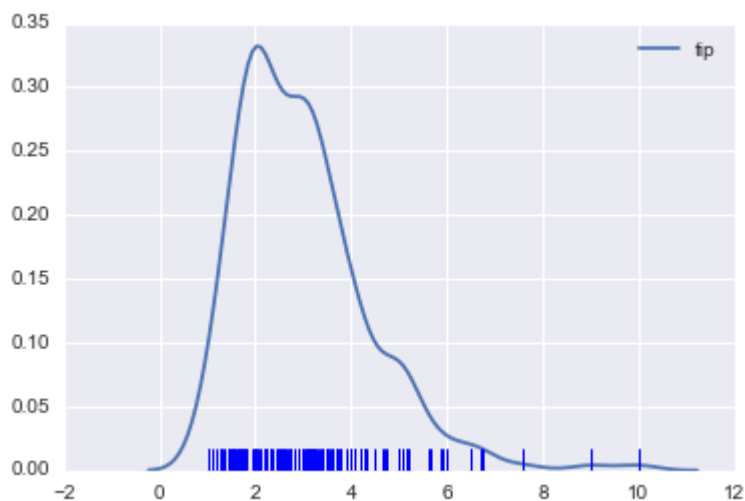
```
Out[41]: <matplotlib.axes._subplots.AxesSubplot at 0x121b82c50>
```



```
In [42]: sns.kdeplot(tips['tip'])  
sns.rugplot(tips['tip'])
```

```
/Users/marci/anaconda/lib/python3.5/site-packages/statsmodels/nonparametric/kde  
tools.py:20: VisibleDeprecationWarning: using a non-integer number instead of a  
n integer will result in an error in the future  
y = X[:m/2+1] + np.r_[0,X[m/2+1:],0]*1j
```

```
Out[42]: <matplotlib.axes._subplots.AxesSubplot at 0x12252cfd0>
```



Great Job!

